Fire-Sale Cascade Simulator :אפיון · Case 02

פרק 2 (אפיון) בתבנית תיק הפרויקט של מגשימים: פיצ'רים, טכנולוגיות, תרשים זרימה ומבנה בסיס נתונים. המסמך כתוב כך שמפתח אחר יוכל לבנות את המערכת ממנו. כל פיצ'ר מפורט ברמת "קוד במילים": מה הוא עושה, איך הוא עובד שלב אחר שלב, אילו פונקציות ומבני נתונים, אילו כלים מהתעשייה משלימים אותו, ומה הקלט והפלט שלו.

סקירת המערכת

מה המערכת עושה, במשפט אחד

Fire-Sale Cascade Simulator בונה שוק פיננסי מדומה מסוכנים עם מאזנים אמיתיים (קרנות פנסיה ממונפות, בנקים, קרנות אשראי, מבטחים), מזריק לו זעזוע, ומראה סבב אחר סבב מי נאלץ למכור, איך המכירה שלו מזיזה מחירים, ומי נופל אחריו.

מה השתנה מאז ה-POC ומודל האיום

- ה-POC עבד עם סוג סוכן אחד, נכס אחד, וכלל תגובה אחד. המערכת המלאה צריכה לשחזר גם את משבר ה-LDI (מנוף ודרישות בטוחות) וגם את SVB (ריצת מפקידים על בנק עם הפסדים לא ממומשים). אלה שני מנגנונים שונים, ולכן נדרשים כמה סוגי סוכנים וכמה כללי תגובה.
- ה-POC חשף שהתוצאה קופצת מ"בטוח" ל"קריסה" בטווח צר של פרמטר אחד. לכן המערכת לא מציגה מספר בודד: כל תוצאה יוצאת כטווח, עם ניתוח רגישות שמראה איזה פרמטר אחראי לאי-הוודאות (פיצ'ר 8).
- **מודל האיום קבע שני דברים:** האיום המרכזי הוא "הנדסת תוצאה" מפחידה, ונתונים אמיתיים שיוזנו לכלי הופכים אותו למטרה רגישה. לכן כל החישוב רץ אצל המשתמש בלבד (פיצ'ר 10), ועולמות "רשמיים" חתומים (פיצ'ר 1).

רשימת הפיצ'רים

- פיצ'ר 1, בניית עולם וספריית סוכנים (World Builder & Agent Library)
- פיצ'ר 2, מודל מאזן ושערוך (Balance Sheet & Valuation)
- פיצ'ר 3, מנוע כללי תגובה (Behavioral Rules Engine)
- פיצ'ר 4, מודל השפעת מחיר ונזילות (Market Impact Model)
- פיצ'ר 5, מנוע המפל (Cascade Engine)
- פיצ'ר 6, רשת חשיפות והדבקה ישירה (Exposure Network & Contagion)
- פיצ'ר 7, התערבות ומדיניות (Intervention Module)
- פיצ'ר 8, סריקת אי-ודאות וניתוח רגישות (Uncertainty Sweep & Sensitivity)
- פיצ'ר 9, שחזור משברים וכיול (Historical Replay & Calibration)
- פיצ'ר 10, תצוגת המפל וקישור משחזר (Cascade View & Reproducible Links)

פיצ'ר 1: בניית עולם וספריית סוכנים (World Builder & Agent Library)

מה הפיצ'ר עושה?

יוצר את "העולם" שעליו רצה הסימולציה: רשימת סוכנים, לכל אחד סוג, מאזן, ומגבלות. העולם יכול להיבנות משלוש דרכים: מעולם מוכן מהספרייה (למשל "בריטניה, ספטמבר 2022"), מהתפלגויות שהמשתמש מגדיר ("60 קרנות פנסיה, מנוף ממוצע בין 3 ל-5"), או מטבלה שהמשתמש טוען בעצמו.

איך הפיצ'ר עובד?

סוגי הסוכנים

כל סוג סוכן מוגדר בקובץ YAML שמתאר את מבנה המאזן שלו ואת כללי התגובה שלו (פיצ'ר 3). חמישה סוגים מובנים:

- **LDI Pension Fund:** מחזיקה אג"ח ארוכות כדי לגדר התחייבויות פנסיה, וממנפת את עצמה דרך repo ונגזרי ריבית. כשהתשואות עולות, הנגזרים מפסידים ונדרשות בטוחות.
- **Commercial Bank:** מחזיק אג"ח בשני תיקים, HTM (מוחזק לפדיון, לא משוערך בדוחות) ו-AFS (זמין למכירה, משוערך), וממומן בפיקדונות, חלקם מבוטחים וחלקם לא.
- **Private Credit Fund:** נותנת הלוואות בריבית משתנה, ממומנת בהון וקווי אשראי. חשופה לחדלות פירעון של הלווים כשהריבית עולה.
- **Insurer:** מחזיק אג"ח ארוכות, עם התחייבויות ארוכות ומנוף נמוך. משמש לרוב כ"קונה" ולא כמוכר.
- **Money Market Fund:** מחזיקה אג"ח קצרות, חשופה לבקשות פדיון של משקיעים.

שלוש דרכים לבנות עולם

מתבנית: בחירת עולם מוכן מהספרייה, שכולל גם את המקורות שעליהם הוא מבוסס (דוחות יציבות פיננסית של בנקים מרכזיים, דוחות רבעוניים ציבוריים). מהתפלגויות: לכל פרמטר (הון, מנוף, ספי בטוחות, שיעור פיקדונות לא מבוטחים) מוגדרת התפלגות (נורמלית קטומה, לוג-נורמלית, או אחידה), והעולם נדגם עם seed קבוע. מטבלה: CSV עם עמודה לכל שדה מאזן. הטבלה נבדקת מול סכמה, ולעולם אינה נשלחת לשרת (פיצ'ר 10).

עולמות חתומים

מענה לאיום "התחזות לכיול" ממודל האיום: כל עולם בספרייה הרשמית נשמר כקובץ JSON קנוני וחתום ב-Ed25519 של הפרויקט. כשעולם נטען, החתימה נבדקת, והממשק מציג תג "עולם רשמי, גרסה X" או "עולם מותאם אישית". אי אפשר לטעון שתוצאה מבוססת על עולם רשמי אם היא לא.

טכנולוגיות ותשתיות תעשייה משלימות

- **Pydantic ו-JSON Schema:** כל סוג סוכן וכל עולם מוגדרים כמחלקות Pydantic, שמייצרות אוטומטית JSON Schema. התרומה: הטבלה שהמשתמש טוען נבדקת מול אותה סכמה בדפדפן, לפני שהיא נכנסת למנוע.
- **NumPy Generator (PCG64):** מחולל מספרים אקראיים מודרני עם seed. התרומה: אותו seed ייתן תמיד בדיוק אותו עולם, על כל מחשב, וזה הבסיס לשחזור (פיצ'ר 10).
- **Mesa (כהשראה למבנה):** ספריית ABM מובילה בפייתון, שמגדירה Agent ו-Model כמחלקות. אנחנו מאמצים את המבנה המושגי, אבל לא את מימוש "סוכן אחר סוכן" שלה, מסיבות ביצועים (פיצ'ר 5).

מבנה נתונים

```
class AgentType(BaseModel):
    type_id: Literal["ldi", "bank", "pcf", "insurer", "mmf"]
    balance_fields: list[str]        # which BS lines exist for this type
    rule_set_id: str                 # -> RuleSet (feature 3)

class WorldSpec(BaseModel):
    world_id: str                    # sha256 of canonical spec
    name: str
    seed: int
    populations: list[Population]    # type_id, count, distributions
    assets: list[AssetSpec]          # gilts_long, gilts_short, corp_bonds, loans
    sources: list[str]               # where each preset number came from
```

```
    official: bool
    signature: bytes | None          # Ed25519, official worlds only
```

קלט ופלט

קלט: עולם מהספרייה, הגדרת התפלגויות, או קובץ CSV. פלט: WorldSpec מאומת, ומערכי המאזנים של כל הסוכנים (פיצ'ר 2).

ספריות ופונקציות מרכזיות

שימוש	ספרייה	פעולה
BaseModel(), model_json_schema	Pydantic	הגדרת סכמות
default_rng(seed), lognormal(), truncated normal	NumPy	דגימה
VerifyKey.verify() בשרת, nacl.sign.detached.verify בדפדפן	PyNaCl / tweetnacl-js	בדיקת חתימה

פיצ'ר 2: מודל מאזן ושערוך (Balance Sheet & Valuation)

מה הפיצ'ר עושה?

מחזיק את המאזן של כל סוכן ומשערך אותו מחדש בכל פעם שמחירים זזים. בלי שערוך מדויק, אין משמעות לשאלה "מי עבר את הסף": ההבדל בין מנוף 4.9 ל-5.1 הוא ההבדל בין שקט לספירלה, כפי שה-POC הראה.

איך הפיצ'ר עובד?

שורות המאזן

כל סוכן מחזיק וקטור של אחזקות (כמות יחידות מכל נכס) ושורות התחייבויות: repo (הלוואה מגובה בטוחות), פיקדונות מבוטחים, פיקדונות לא מבוטחים, התחייבויות פנסיה, הון. בנוסף נשמרת "פוזיציית נגזרים" בערך נקודתי (DV01, כמה הנגזר מרוויח או מפסיד לכל נקודת בסיס בתשואה), כי זה מה שמייצר את דרישות הבטוחות של קרנות ה-LDI.

שערוך אג"ח

ה-POC השתמש בקירוב duration בלבד. המערכת משתמשת בקירוב מסדר שני: שינוי המחיר באחוזים שווה למינוס duration כפול שינוי התשואה, ועוד חצי convexity כפול שינוי התשואה בריבוע. הקירוב הזה מדויק בתנועות של כמה נקודות אחוז, והוא וקטורי. לבדיקה, אפשר להפעיל שערוך מלא: היוון כל תזרימי האג"ח לפי עקום NSS (אותו עקום כמו בסימולטור החוב), והשוואה לקירוב.

HTM מול AFS: הלקח של SVB

אג"ח בתיק HTM לא משוערכות בדוחות החשבונאיים, ולכן ההון "על הנייר" לא יורד כשהתשואות עולות. אבל אם הבנק נאלץ למכור אותן, ההפסד מתממש בבת אחת. המודל מחזיק לכל בנק שני מספרי הון: הון חשבונאי (בלי הפסדי HTM) והון כלכלי (כולל אותם). כללי התגובה של מפקידים (פיצ'ר 3) מגיבים להון הכלכלי, כי זה מה שהשוק רואה. זה בדיוק מה שקרה ל-SVB ב-2023.

תספורות על בטוחות

כשסוכן מעמיד נכס כבטוחה ל-repo, המלווה לא סופר את מלוא ערכו אלא מוריד "תספורת" (haircut), למשל 2% לאג"ח ממשלתית קצרה ו-5% לארוכה. בזמן לחץ התספורות עולות, וזה מנגנון נוסף שמייצר דרישות בטוחות גם בלי ירידת מחיר. התספורות מוגדרות לכל נכס כפונקציה של התנודתיות האחרונה.

טכנולוגיות ותשתיות תעשייה משלימות

- **NumPy (Structure of Arrays):** מטריצת אחזקות בגודל מספר הסוכנים כפול מספר הנכסים, וקטור מחירים, ומכפלת מטריצה-וקטור אחת נותנת את ערך הנכסים של כל הסוכנים. התרומה: שערוך של אלף סוכנים במיקרו-שניות.
- **QuantLib (לבדיקה בלבד):** שערוך מלא של אג"ח לפי עקום, לבדיקה שהקירוב מסדר שני מדויק מספיק בטווח התנועות שהמערכת מדמה.
- **מושגי Basel III (LCR, Leverage Ratio):** יחסי הנזילות והמינוף הרגולטוריים של בנקים. התרומה: כללי התגובה של בנקים (פיצ'ר 3) כתובים במונחים שאנליסט פיקוח מכיר, ולא במונחים שהמצאנו.

מבנה נתונים

```
class Balances:                  # Structure of Arrays, N agents x K assets
    holdings: np.ndarray         # (N, K) units held
    is_htm: np.ndarray           # (N, K) bool, banks only
    repo: np.ndarray             # (N,) secured funding
    dep_insured: np.ndarray      # (N,)
    dep_uninsured: np.ndarray    # (N,)
    pension_liab: np.ndarray     # (N,) PV of liabilities
    dv01_deriv: np.ndarray       # (N,) derivative P&L per bp
    cash: np.ndarray             # (N,)

class AssetState:                # K assets
    price: np.ndarray            # (K,)
    duration: np.ndarray         # (K,)
    convexity: np.ndarray        # (K,)
    haircut: np.ndarray          # (K,)
```

קלט ופלט

קלט: WorldSpec ומחירי הנכסים הנוכחיים. פלט: לכל סוכן, ערך נכסים, הון חשבונאי, הון כלכלי, מנוף, ופער בטוחות.

ספריות ופונקציות מרכזיות

שימוש	ספרייה	פעולה
holdings @ price, np.maximum	NumPy	שערוך וקטורי
FixedRateBond, DiscountingBondEngine	QuantLib	בדיקת שערוך

פיצ'ר 3: מנוע כללי תגובה (Behavioral Rules Engine)

מה הפיצ'ר עושה?

מחליט מה כל סוכן עושה בכל סבב, לפי המצב שלו. הכללים כתובים כנתונים (YAML) ולא כקוד, כך שאפשר לראות, לשנות ולהשוות אותם. זה קריטי כי, כפי שנכתב במחקר, כללים פשטניים הם אחת המגבלות הגדולות של המודל, והם חייבים להיות שקופים.

איך הפיצ'ר עובד?

מבנה כלל

כל כלל מורכב מתנאי, פעולה, ועדיפות. התנאי הוא ביטוי על שדות המאזן (למשל leverage > lmax, או collateral_gap > cash). הפעולה היא אחת מרשימה סגורה: SELL_TO_TARGET (מכור עד שהמנוף חוזר ליעד), POST_COLLATERAL (העמד מזומן או נכסים), WITHDRAW (מפקידים מושכים), REDEEM (משקיעים מבקשים פדיון), DEFAULT (חדלות פירעון). העדיפות קובעת מה קורה כשכמה כללים מתקיימים יחד.

הכללים של כל סוג

- **LDI:** כשה-DV01 של הנגזרים מייצר הפסד, נדרשות בטוחות. אם יש מזומן, משתמשים בו. אם לא, מוכרים אג"ח ארוכות. אם ההון נמחק, DEFAULT.
- **Bank:** מפקידים לא מבוטחים מושכים לפי פונקציה של ההון הכלכלי: שיעור המשיכה בסבב שווה לפונקציה לוגיסטית של (סף פחות יחס ההון הכלכלי), עם פרמטר "רגישות" שמייצג כמה מהר השמועה מתפשטת. כשהמשיכות עולות על המזומן, הבנק מוכר AFS ואחר כך HTM, ומממש את ההפסד.
- **Private Credit Fund:** כשהריבית עולה, חלק מהלווים לא עומדים בתשלום, לפי מקדם כיסוי ריבית (ICR) מדומה. הפסדים מורידים הון, ומשקיעים מבקשים פדיון.
- **Insurer:** קונה אג"ח ארוכות כשהתשואה עוברת סף, עד גבול תיאבון הסיכון שלו. זה ה"שובר גלים" הטבעי בשוק.
- **Money Market Fund:** משקיעים מבקשים פדיון לפי שיעור קבוע או לפי ירידת הערך, והקרן מוכרת אג"ח קצרות.

סדר מכירה (liquidity waterfall)

כשסוכן צריך מזומן, הוא מוכר לפי סדר מוגדר: קודם הנכס הנזיל ביותר, אחר כך הבא. הסדר הזה חשוב: הוא קובע לאיזה שוק המכירה תגיע, ולכן מי ייפגע ממנה. בנק שמוכר קודם אג"ח קצרות פוגע בקרנות כספיות. קרן פנסיה שמוכרת אג"ח ארוכות פוגעת בקרנות פנסיה אחרות.

טכנולוגיות ותשתיות תעשייה משלימות

- **YAML עם JSON Schema:** הכללים נשמרים כטקסט קריא ונבדקים מול סכמה. התרומה: אנליסט יכול לקרוא כלל בלי לקרוא פייתון.
- **numexpr (בשרת) או מהדר ביטויים פשוט משלנו (בדפדפן):** ממיר ביטוי כמו "leverage > lmax" לפעולה וקטורית על כל הסוכנים בבת אחת. התרומה: כללים שהם נתונים, בלי לשלם על זה בביצועים. בדפדפן, ספריית ביטויים קטנה שמתרגמת לפעולות NumPy, בלי eval על קלט של משתמש.
- **מודל ריצת המפקידים של Diamond ו-Dybvig (1983):** המודל הקלאסי של ריצה על בנקים, שבגללו מחבריו קיבלו פרס נובל לכלכלה ב-2022. התרומה: כלל המשיכה שלנו הוא גרסה דינמית ופשוטה של הרעיון המרכזי שלו: כל מפקיד מושך כי הוא חושש שאחרים ימשכו.

מבנה נתונים

```
rule_set: ldi_v1
rules:
  - id: LDI_MARGIN
    when: "collateral_gap > 0"
    then: POST_COLLATERAL
    priority: 1
  - id: LDI_FIRE_SALE
    when: "collateral_gap > cash"
    then: {action: SELL_TO_TARGET, order: [gilts_short, gilts_long]}
    priority: 2
  - id: LDI_DEFAULT
```

```
    when: "equity_economic <= 0"
    then: DEFAULT
    priority: 0
```

קלט ופלט

קלט: מצב המאזנים אחרי שערוך, ו-RuleSet לכל סוג. פלט: לכל סוכן, הפעולה שנבחרה, וכמות המכירה בכל נכס.

ספריות ופונקציות מרכזיות

פעולה	ספרייה	שימוש
קריאת כללים	PyYAML, jsonschema	()safe_load(), validate
הערכה וקטורית	numexpr	evaluate("leverage > lmax", local_dict)
פונקציית ריצה	NumPy	(exp(-k * (thr - ratio)) + 1) / 1

פיצ'ר 4: מודל השפעת מחיר ונזילות (Market Impact Model)

מה הפיצ'ר עושה?

מתרגם "כמה מכרו" ל"כמה המחיר זז". זה המקום שבו המכירה של סוכן אחד הופכת לבעיה של סוכן אחר, ולכן זה החלק הרגיש ביותר במודל. ה-POC השתמש בפונקציה ליניארית אחת. המערכת תומכת בכמה מודלים, ומראה למשתמש כמה התוצאה תלויה בבחירה ביניהם.

איך הפיצ'ר עובד?

שלושה מודלים

- **ליניארי (Kyle's lambda):** שינוי המחיר פרופורציוני לכמות שנמכרה. פשוט, ומתאים לתנועות קטנות. נקרא על שם המודל של Albert Kyle מ-1985.
- **שורש ריבועי (square-root law):** שינוי המחיר פרופורציוני לשורש של הכמות, ביחס לנפח המסחר היומי. זה החוק האמפירי המוכר ביותר בתחום, שנמדד שוב ושוב בשווקים שונים. המשמעות: מכירה כפולה לא מזיזה את המחיר פי שניים, אלא פי 1.4 בערך.
- **עם קיבולת דילרים:** לדילרים (בנקים שעושים שוק) יש מאזן מוגבל. עד גבול מסוים הם סופגים מכירות בהשפעה מתונה, ומעבר לו ההשפעה קופצת. זה משקף את מה שקרה בבריטניה ב-2022: השוק עבד, עד שפתאום לא.

השפעה צולבת

מכירת אג"ח ארוכות מזיזה גם את האג"ח הבינוניות, כי משקיעים מחליפים ביניהן. מטריצת השפעה צולבת (cross-impact) בגודל נכסים כפול נכסים קובעת כמה. ברירת המחדל נגזרת מהמתאם ההיסטורי בין תשואות האג"ח.

התאוששות

אחרי סבב מכירות, חלק מההשפעה "מתאושש" בסבב הבא, כשקונים חדשים נכנסים (למשל מבטחים, פיצ'ר 3). קצב ההתאוששות הוא פרמטר: מחיר שלא מתאושש בכלל מייצר ספירלה מהר יותר.

טכנולוגיות ותשתיות תעשייה משלימות

- **מודלי market impact של Almgren ו-Chriss:** המסגרת הסטנדרטית בתעשייה להפרדה בין השפעה זמנית לקבועה, שמשמשת כל שולחן מסחר אלגוריתמי. התרומה: ההפרדה שלנו בין השפעה להתאוששות נשענת על מסגרת מוכרת.
- Schaanning, "Fire Sales, Indirect Contagion and Systemic Stress Testing" ו-Cont: עבודה שמגדירה את ההדבקה דרך "אחזקות משותפות": שני סוכנים שלא חייבים זה לזה כלום, אבל מחזיקים את אותו נכס, נפגעים זה מזה דרך המחיר. התרומה: המודל שלנו ממדל במפורש את הערוץ הזה.

מבנה נתונים

```
ImpactModel {
  kind: Literal["linear", "sqrt", "dealer_capacity"]
  lam: np.ndarray            # (K,) per-asset impact coefficient
  adv: np.ndarray            # (K,) average daily volume, for sqrt law
  dealer_cap: np.ndarray     # (K,) dealer absorption capacity
  cross: np.ndarray          # (K, K) cross-impact matrix, diag = 1
  recovery: float            # share of impact that decays per round
}
```

קלט ופלט

קלט: וקטור המכירות נטו בכל נכס בסבב, ומצב הנכסים. פלט: וקטור שינויי מחיר, ושינוי התשואה המקביל לכל אג"ח.

ספריות ופונקציות מרכזיות

פעולה	ספרייה	שימוש
השפעה וקטורית	NumPy	sign(q) * lam * sqrt(abs(q) / adv)
השפעה צולבת	NumPy	cross @ own_impact

פיצ'ר 5: מנוע המפל (Cascade Engine)

מה הפיצ'ר עושה?

זו הליבה הטכנולוגית. המנוע מריץ את הסבבים: שערוך, החלטות, מכירות, השפעת מחיר, וחוזר, עד שהשוק נרגע או שעובר מספר הסבבים המקסימלי. הפלט הוא ההיסטוריה המלאה של כל סבב.

איך הפיצ'ר עובד?

לולאת הסבב

- **שערוך:** כל המאזנים משוערכים לפי המחירים הנוכחיים (פיצ'ר 2).
- **החלטה:** מנוע הכללים מחליט לכל סוכן מה לעשות (פיצ'ר 3).
- **סליקה:** מכירות מומרות למזומן במחיר הנוכחי פחות השפעה זמנית, מזומן משלם repo ומשיכות, וסוכנים שהגיעו ל-DEFAULT מוציאים את כל הנכסים שלהם לשוק.
- **התערבות:** אם מוגדרת, הבנק המרכזי או מנגנון אחר סופג חלק מהמכירות (פיצ'ר 7).
- **השפעת מחיר:** המכירות נטו שנותרו מזיזות מחירים (פיצ'ר 4).
- **הדבקה ישירה:** חדלות פירעון של סוכן מפילה ערך של הלוואות שאחרים נתנו לו (פיצ'ר 6).
- **רישום:** כל מצב נשמר כ-RoundState.

תנאי עצירה

המנוע עוצר כשהמכירות הכפויות בסבב קטנות מסף (0.01% מסך הנכסים בשוק), כששינוי המחירים בסבב זניח, או אחרי 100 סבבים. אם עצר בגלל המגבלה ולא בגלל התכנסות, התוצאה מסומנת "לא התכנס", כי זה סימן שהמודל בספירלה שלא נגמרת.

בדיקות שימור

בכל סבב: סך המזומן במערכת, פלוס ערך הנכסים, פחות ההפסדים שנרשמו, חייב להתאים לסבב הקודם. כמות היחידות שנמכרו שווה לכמות שנקנתה (על ידי הבנק המרכזי, המבטחים, או "השוק" המופשט שמאחורי מודל ההשפעה). הפרה עוצרת את ההרצה עם שגיאה.

סבבים ושעון אמיתי

סבב הוא לא יום. במשבר ה-LDI הספירלה התפתחה במשך כמה ימים, ובמקרה של SVB תוך יום וחצי. לכל עולם בספרייה מוגדר "אורך סבב משוער", כדי שהתצוגה תוכל לומר "בערך יומיים" ולא רק "6 סבבים", עם הסתייגות גלויה שזה אומדן.

טכנולוגיות ותשתיות תעשייה משלימות

- **NumPy (וקטוריזציה מלאה):** ה-POC מדד 0.28 שניות ל-1,000 הרצות של 60 סוכנים. המערכת המלאה, עם 5 סוגים ו-4 נכסים, צפויה להיות איטית פי כמה, ועדיין מתחת לשנייה לסריקה של מאות עולמות.
- **Numba (בשרת, לכיול):** לכיול (פיצ'ר 9) נדרשים עשרות אלפי הרצות. הידור לולאת הסבבים מאיץ אותה פי עשרות.
- **Pyodide עם Web Worker:** הרצה בדפדפן, ברקע, בלי לתקוע את הממשק.

מבנה נתונים

```
RoundState {
  round: int
  prices: np.ndarray            # (K,)
  yields_bp: np.ndarray         # (K,) change vs start
  forced_sales: np.ndarray      # (N, K)
  cb_absorbed: np.ndarray       # (K,)
  equity_econ: np.ndarray       # (N,)
  leverage: np.ndarray          # (N,)
  status: np.ndarray            # (N,) int8: 0 ok, 1 acting, 2 default
}

CascadeResult {
  run_id: str
  rounds: list[RoundState]
  converged: bool
  amplification: float          # final yield move / initial shock
  share_forced: float
  defaults: int
}
```

קלט ופלט

קלט: עולם, זעזוע (שינוי תשואה התחלתי, או משיכת פיקדונות התחלתית), והתערבות אופציונלית. פלט: CascadeResult.

ספריות ופונקציות מרכזיות

פעולה	ספרייה	שימוש
חישוב וקטורי	NumPy	masking, np.add.reduce
האצה בשרת	Numba	njit(cache=True)@
ריצה בדפדפן	Pyodide	runPythonAsync בתוך Worker

פיצ'ר 6: רשת חשיפות והדבקה ישירה (Exposure Network & Contagion)

מה הפיצ'ר עושה?

יש שני ערוצים שבהם סוכן אחד מפיל אחר. הראשון, דרך מחירים (פיצ'ר 4): מכרתי, המחיר ירד, ואתה מפסיד. השני, ישיר: הלוויתי לך, נפלת, ואני מפסיד את ההלוואה. הפיצ'ר הזה ממדל את הערוץ השני, ומזהה אילו סוכנים הם "צמתים מרכזיים" שהנפילה שלהם מפילה הכי הרבה.

איך הפיצ'ר עובד?

הרשת

גרף מכוון שבו כל צלע היא הלוואה: מי הלווה למי וכמה (repo, הלוואות בין-בנקאיות, קווי אשראי לקרנות). בעולמות מהתפלגויות, הגרף נוצר אקראית לפי מודל "core-periphery": מעט בנקים גדולים במרכז שמלווים זה לזה ולכולם, והרבה סוכנים קטנים בפריפריה. זה המבנה שנמדד בפועל ברשתות בין-בנקאיות.

הדבקה

כשסוכן מגיע ל-DEFAULT, כל מי שהלווה לו מקבל הפסד בגובה ההלוואה כפול (1 פחות שיעור ההשבה). שיעור ההשבה הוא פרמטר (ברירת מחדל 40%). ההפסד מוריד הון, ויכול לגרום לסוכן הבא לעבור סף, להתחיל למכור, או ליפול בעצמו.

DebtRank

כדי לזהות צמתים מרכזיים, הפיצ'ר מחשב DebtRank לכל סוכן: אלגוריתם מ-2012 (Battiston ועמיתים) שמודד איזה חלק מהערך הכלכלי ברשת ייפגע אם הסוכן הזה יאבד את ההון שלו. האלגוריתם דומה ל-PageRank של גוגל, אבל עם מנגנון שמונע ספירה כפולה של אותו הפסד במעגלים ברשת. התוצאה: דירוג של מי "מסוכן מערכתית" בתוך העולם המדומה.

טכנולוגיות ותשתיות תעשייה משלימות

- **NetworkX:** הספרייה הסטנדרטית בפייתון לגרפים. התרומה: בניית הרשת, חישוב מדדי מרכזיות להשוואה, וייצוא לתצוגה.
- **SciPy sparse:** הרשת דלילה (רוב הסוכנים לא מלווים לרוב הסוכנים), ולכן היא נשמרת כמטריצה דלילה. התרומה: חישוב ההדבקה הוא מכפלת מטריצה דלילה בווקטור, מהיר גם לאלפי סוכנים.
- **DebtRank (Battiston et al., Scientific Reports, 2012):** מדד הסיכון המערכתי שהבנק המרכזי האירופי ובנקים מרכזיים נוספים בחנו בעבודות מחקר. התרומה: "מי הכי מסוכן" נמדד באלגוריתם מוכר ולא בתחושה.

מבנה נתונים

```
ExposureGraph {
  lender: np.ndarray        # (E,) agent index
  borrower: np.ndarray      # (E,)
  amount: np.ndarray        # (E,)
```

```
  kind: np.ndarray          # (E,) int8: repo, interbank, credit_line
  recovery: float
}
DebtRankResult { agent_index: int; debtrank: float; rank: int }
```

קלט ופלט

קלט: רשימת הלוואות (או הגדרת core-periphery), ורשימת הסוכנים שנפלו בסבב. פלט: הפסדי הדבקה לכל סוכן, ו-DebtRank לכל סוכן.

ספריות ופונקציות מרכזיות

פעולה	ספרייה	שימוש
גרף	NetworkX	DiGraph, from_scipy_sparse_array
הדבקה	SciPy	sparse.csr_matrix, dot
מרכזיות	מימוש משלנו	DebtRank לפי המאמר המקורי

פיצ'ר 7: התערבות ומדיניות (Intervention Module)

מה הפיצ'ר עושה?

ה-POC הראה שהתערבות לא מונעת מכירות, היא שוברת את המעגל. הפיצ'ר מאפשר לבדוק סוגים שונים של התערבות, מתי הם מופעלים, ובאיזה גודל, ולהשוות ביניהם על אותו עולם.

איך הפיצ'ר עובד?

ארבעה כלים

- **רכישות של בנק מרכזי:** הבנק המרכזי קונה אג"ח מסוג מסוים, עד תקרה ליום ותקרה כוללת, לתקופה מוגבלת. בהשראת ההתערבות של ה-Bank of England בספטמבר 2022: הוא הודיע על רכישות זמניות של אג"ח ארוכות עד 5 מיליארד לישי"ט ביום למשך 13 ימי עסקים, ובפועל רכש כ-19 מיליארד בלבד. ההודעה עצמה הרגיעה את השוק.
- **הקלה בבטוחות:** הורדת תספורות או הארכת מועד לדרישות בטוחות.
- **ביטוח פיקדונות מורחב:** כל הפיקדונות מבוטחים, ולכן מפקידים לא מושכים. בהשראת ההחלטה ב-SVB, מרץ 2023.
- **מסגרת הלוואות לפי ערך נקוב:** בנקים יכולים ללוות מהבנק המרכזי כנגד אג"ח לפי הערך הנקוב שלהן ולא לפי מחיר השוק, וכך לא נאלצים למכור בהפסד. בהשראת ה-Bank Term Funding Program (BTFP) שהפד הקים במרץ 2023.

טריגרים

כל התערבות מופעלת לפי טריגר: מסבב מסוים, או כשמדד עובר סף (למשל, תנועת תשואה של יותר מ-100 נקודות בסיס). כך אפשר לבדוק שאלה כמו "מה אם הבנק המרכזי היה מגיב סבב אחד מוקדם יותר".

ההשפעה של ההודעה

במציאות, חלק מההשפעה של התערבות מגיע מההודעה עצמה, לפני רכישה אחת. הפיצ'ר ממדל את זה כפרמטר "אמינות": הכרזה אמינה מורידה את מקדם ההשפעה (פיצ'ר 4) כבר מהסבב שבו היא מוכרזת. הפרמטר מסומן בבירור כהנחה.

טכנולוגיות ותשתיות תעשייה משלימות

- **תיעוד רשמי של ההתערבויות:** הודעות ה-Bank of England מספטמבר 2022, ותנאי ה-BTFP של הפד. התרומה: כל כלי בפיצ'ר מבוסס על התערבות שקרתה באמת, עם תנאים מתועדים.
- **Strategy pattern:** דפוס תכנון שבו כל התערבות היא מחלקה עם אותו ממשק ((apply(state מחזיר state). התרומה: הוספת סוג התערבות חדש בלי לגעת במנוע.

מבנה נתונים

```
class Intervention(Protocol):
    def triggered(self, state: RoundState) -> bool: ...
    def apply(self, state: RoundState, sales: np.ndarray) -> np.ndarray: ...

CBPurchase { asset: int; daily_cap: float; total_cap: float; rounds: int; credibility:
float }
CollateralRelief { haircut_cut: float; grace_rounds: int }
DepositGuarantee { coverage: Literal["insured_only", "all"] }
ParValueFacility { eligible_assets: list[int]; rate_spread_bp: float }
```

קלט ופלט

קלט: רשימת התערבויות עם טריגרים. פלט: מכירות נטו אחרי ספיגה, ומדדי עלות לכל התערבות (כמה נרכש, כמה הולווה).

ספריות ופונקציות מרכזיות

שימוש	ספרייה	פעולה
Intervention	typing.Protocol	ממשק אחיד
הרצה מקבילה על אותו עולם	concurrent.futures	השוואת התערבויות

פיצ'ר 8: סריקת אי-ודאות וניתוח רגישות (Uncertainty Sweep & Sensitivity)

מה הפיצ'ר עושה?

זה הפיצ'ר שה-POC הפך לחובה. במקום "התשואה תזנק ב-1.13 נקודות אחוז", המערכת אומרת "בין 0.8 ל-1.6, ו-70% מאי-הוודאות הזו נובעת ממנוף הקרנות, שאיננו יודעים". זה גם המענה המרכזי לאיום "הנדסת תוצאה": תוצאה מפחידה תמיד מוצגת יחד עם הטווח ועם הפרמטר שאחראי לה.

איך הפיצ'ר עובד?

סריקה

המשתמש מסמן אילו פרמטרים לא ידועים, ונותן לכל אחד טווח. הפיצ'ר מייצר N עולמות (ברירת מחדל 512) בדגימת Sobol, שמכסה את מרחב הפרמטרים באופן אחיד יותר מדגימה אקראית. כל עולם מורץ במנוע המפל. הפלט: התפלגות של כל מדד תוצאה, עם אחוזונים 10, 50 ו-90.

ניתוח רגישות גלובלי

מדדי Sobol מפרקים את השונות של התוצאה לפי פרמטרים: S1 הוא החלק שכל פרמטר מסביר לבד, ו-ST הוא החלק שהוא מסביר כולל אינטראקציות עם אחרים. התצוגה היא "תרשים טורנדו": הפרמטרים ממוינים לפי ST. אם ST של המנוף הוא 0.7, אז 70% מאי-הוודאות בתוצאה תלויה בו.

מציאת נקודת המפנה

ה-POC גילה סף: בין מנוף 4 ל-5.5 התוצאה קופצת. הפיצ'ר מוצא את הסף הזה אוטומטית: לכל פרמטר, חיפוש בינארי על הערך שבו ההסתברות לספירלה (תנועה מעל סף מוגדר) עוברת 50%. התוצאה מוצגת כמשפט: "מעל מנוף ממוצע של 4.7, ספירלה סבירה יותר מאשר לא".

ברירת מחדל שאי אפשר לעקוף

תוצאה של עולם יחיד מוצגת תמיד עם תווית "הרצה בודדת, ראו טווח", וקישור לסריקה של אותו עולם עם הטווחים ברירת המחדל. אי אפשר להוריד תמונה של הרצה בודדת בלי שהתווית כלולה בה.

טכנולוגיות ותשתיות תעשייה משלימות

- **SALib:** הספרייה הפתוחה הנפוצה בפייתון לניתוח רגישות, עם מימוש של שיטת Saltelli לדגימה ושל מדדי Sobol. התרומה: מדדי רגישות שמחושבים לפי השיטה המקובלת במחקר, ולא לפי שיטה שהמצאנו.
- **scipy.stats.qmc.Sobol:** דגימה קוואזי-אקראית. התרומה: מניפה יציבה עם פחות הרצות.
- **Uncertainty Quantification (UQ) כתחום:** התחום ההנדסי שעוסק בדיוק בזה, בסימולציות של אקלים, תעופה וגרעין. התרומה: אנחנו לא ממציאים את הגישה, אנחנו מיישמים תחום מבוסס על בעיה פיננסית.

מבנה נתונים

```
SweepSpec { world_id; params: list[ParamRange]; n: int; seed: int; spiral_threshold_bp:
float }
ParamRange { name: str; low: float; high: float; dist: Literal["uniform", "loguniform"] }
SweepResult {
  sweep_id: str
  outcome_quantiles: dict[str, tuple[float, float, float]]   # p10, p50, p90
  p_spiral: float
  sobol: dict[str, tuple[float, float]]                      # S1, ST
  tipping_points: dict[str, float | None]
}
```

קלט ופלט

קלט: עולם, רשימת פרמטרים עם טווחים. פלט: SweepResult עם טווחים, הסתברות לספירלה, מדדי רגישות ונקודות מפנה.

ספריות ופונקציות מרכזיות

שימוש	ספרייה	פעולה
sample.saltelli / sample.sobol	SALib	דגימה
analyze.sobol	SALib	רגישות
optimize.bisect על p_spiral(x) - 0.5	SciPy	נקודת מפנה

פיצ'ר 9: שחזור משברים וכיול (Historical Replay & Calibration)

מה הפיצ'ר עושה?

כדי שמישהו יסמוך על הסימולציה, היא צריכה להראות שהיא מתנהגת כמו המציאות במקרים שאנחנו מכירים. הפיצ'ר מכייל את הפרמטרים כך שהמודל ישחזר שני משברים אמיתיים, ומציג בגלוי כמה טוב (או רע) השחזור.

איך הפיצ'ר עובד?

שני מקרי הבוחן

- **בריטניה, ספטמבר 2022:** נתוני היעד מהדוחות הציבוריים של ה-Bank of England: גודל התנועה בתשואות האג"ח הארוכות בימים שאחרי התקציב, היקף הרכישות בפועל, וההתאוששות אחרי ההכרזה.
- **SVB, מרץ 2023:** נתוני היעד מהדוחות הציבוריים של הבנק ושל הפד: חלק הפיקדונות הלא מבוטחים (רובם הגדול, כ-90%), ההפסדים הלא ממומשים בתיק HTM, והיקף המשיכות ביום הקריסה (כ-42 מיליארד דולר ביום אחד).

שיטת הכיול

המודל לא נותן התפלגות סגורה לתוצאות, ולכן אי אפשר להשתמש בשיטות סטטיסטיות רגילות. הפיצ'ר משתמש ב-Approximate Bayesian Computation (ABC): דוגמים פרמטרים מהתפלגות מוקדמת, מריצים, ושומרים רק את הדגימות שהתוצאה שלהן קרובה מספיק לנתוני היעד. מה שנשאר הוא התפלגות של פרמטרים "שמסבירים" את המשבר. בגרסה המתקדמת (ABC-SMC) הסף מצטמצם בהדרגה, וזה מצמצם את מספר ההרצות.

דוח התאמה

לכל מקרה בוחן מוצג דוח: לכל נתון יעד, הערך האמיתי מול טווח הערכים שהמודל המכויל מייצר. נתון שהמודל לא מצליח לשחזר מסומן באדום, ולא מוסתר. למשל: אם המודל משחזר את גודל התנועה אבל לא את מהירות ההתאוששות, זה כתוב.

אימות צולב

הכיול נעשה על מקרה אחד ונבדק על השני, ככל שהמנגנונים חופפים (למשל, פרמטרי השפעת המחיר). אם פרמטרים שכוילו על בריטניה מייצרים תוצאה סבירה גם בתנאי SVB, יש סיבה לסמוך עליהם יותר.

טכנולוגיות ותשתיות תעשייה משלימות

- **pyabc:** ספרייה פתוחה בפייתון ל-ABC-SMC, שתומכת בהרצה מקבילית. התרומה: כיול לפי שיטה מקובלת, עם התפלגות פרמטרים ולא רק "ערך אחד שעובד".
- **Simulated Method of Moments (כחלופה):** שיטה מקובלת בכלכלה לכיול מודלי סוכנים, שבה מתאימים מומנטים (ממוצע, שונות) של הסימולציה לנתונים. התרומה: בדיקה צולבת לתוצאות ה-ABC.
- **Dask (לכיול בשרת):** ספרייה לחישוב מקבילי על כמה מעבדים או מחשבים. התרומה: עשרות אלפי ההרצות של הכיול רצות במקביל.

מבנה נתונים

```
CalibrationTarget { case_id: Literal["uk_ldi_2022", "svb_2023"]; metric: str; value:
float; tolerance: float; source: str }
CalibrationRun { calib_id; case_id; method: Literal["abc_smc", "smm"]; n_sims: int;
accepted: int; posterior_path: str }
FitReport { calib_id; metric; target; sim_p10; sim_p50; sim_p90; within_tolerance: bool }
```

קלט ופלט

קלט: נתוני יעד ממקורות ציבוריים, והתפלגויות מוקדמות לפרמטרים. פלט: התפלגות פרמטרים מכוילת, ודוח התאמה ציבורי.

ספריות ופונקציות מרכזיות

פעולה	ספרייה	שימוש
כיול	pyabc	ABCSMC(models, priors, distance)

פעולה	ספרייה	שימוש
הרצה מקבילית	Dask	distributed.Client
אחסון התפלגות posterior לכל כיול	Parquet	

פיצ'ר 10: תצוגת המפל וקישור משחזר (Cascade View & Reproducible Links)

מה הפיצ'ר עושה?

הממשק: אנימציה של המפל סבב אחר סבב, מפת הרשת, תרשים הטורנדו של הרגישות, וקישור שמשחזר בדיוק כל הרצה. וחשוב מכל: כל החישוב רץ בדפדפן של המשתמש, ושום נתון מאזן לא עוזב את המחשב שלו.

איך הפיצ'ר עובד?

אנימציית המפל

כל סוכן הוא נקודה, בצבע לפי מצב (שקט, פועל, נפל) ובגודל לפי סך המאזן. בכל סבב הנקודות שמוכרות "פולטות" חלקיקים לכיוון השוק, מחוון התשואה זז, והנקודות שעברו סף מחליפות צבע. מתחת לאנימציה: גרף התשואה לאורך הסבבים, וכיתוב חי שמתאר את הסבב במילים ("סבב 3: 12 קרנות קיבלו דרישת בטוחות, 4 מכרו אג"ח ארוכות"). הכיתוב נגיש לקוראי מסך.

ריצה מקומית בלבד

מענה לאיום "דליפת מאזנים": המנוע רץ ב-Pyodide בתוך Web Worker. טבלה שהמשתמש טוען נקראת ב-FileReader API ישירות לזיכרון הדפדפן, ואין קריאת רשת שנושאת אותה. היסטוריית ההרצות נשמרת ב-IndexedDB המקומי, ומשתמש יכול למחוק אותה בלחיצה. בשרת יש רק קבצים סטטיים: הקוד, הספרייה החתומה, והכיולים.

קישור משחזר

הקישור מכיל את world_id (של עולם רשמי או של הגדרת התפלגויות), ה-seed, הזעזוע, ההתערבויות, וגרסת המנוע, מקודדים בחלק ה-fragment של ה-URL (אחרי #), שהדפדפן לא שולח לשרת. מענה לאיום "תוצאה בלי היסטוריה": מי שפותח את הקישור מריץ בדיוק את אותו חישוב. לעולם שנבנה מטבלה פרטית, הקישור לא נוצר, והממשק מסביר למה.

מגבלות משאבים

סריקה מוגבלת ל-4,096 עולמות בדפדפן, עם הערכת זמן לפני ההרצה ואפשרות לבטל באמצע. מענה לאיום "מיצוי משאבים".

טכנולוגיות ותשתיות תעשייה משלימות

- **D3.js עם d3-force:** סימולציית כוחות לפריסת הרשת, כך שסוכנים מחוברים מתקרבים זה לזה. התרומה: מפת רשת קריאה בלי פריסה ידנית.
- **Canvas 2D:** לאנימציה של מאות נקודות וחלקיקים, שבה SVG היה איטי מדי.
- **IndexedDB:** מסד נתונים בתוך הדפדפן. התרומה: היסטוריית הרצות שנשארת אצל המשתמש בלבד.
- **Content Security Policy:** כותרת HTTP שמגבילה לאן הדף רשאי לשלוח בקשות. התרומה: גם באג או ספרייה זדונית לא יכולים לשלוח את נתוני המאזן החוצה, כי הדפדפן יחסום כל בקשה שאינה לשרת שלנו.

מבנה נתונים

```
RunLink {           # encoded in URL fragment, never sent to server
  world_id: str
  seed: int
  shock: { kind: "yield" | "deposit_run"; size: float }
  interventions: list[dict]
  engine_version: str
}
LocalRun { run_id; created_at; link: RunLink | None; result: CascadeResult; private_world:
bool }
```

קלט ופלט

קלט: CascadeResult, SweepResult, ExposureGraph. פלט: אנימציה, מפת רשת, טורנדו, קישור משחזר, ותמונה להורדה שכוללת תמיד את התוויות וההנחות.

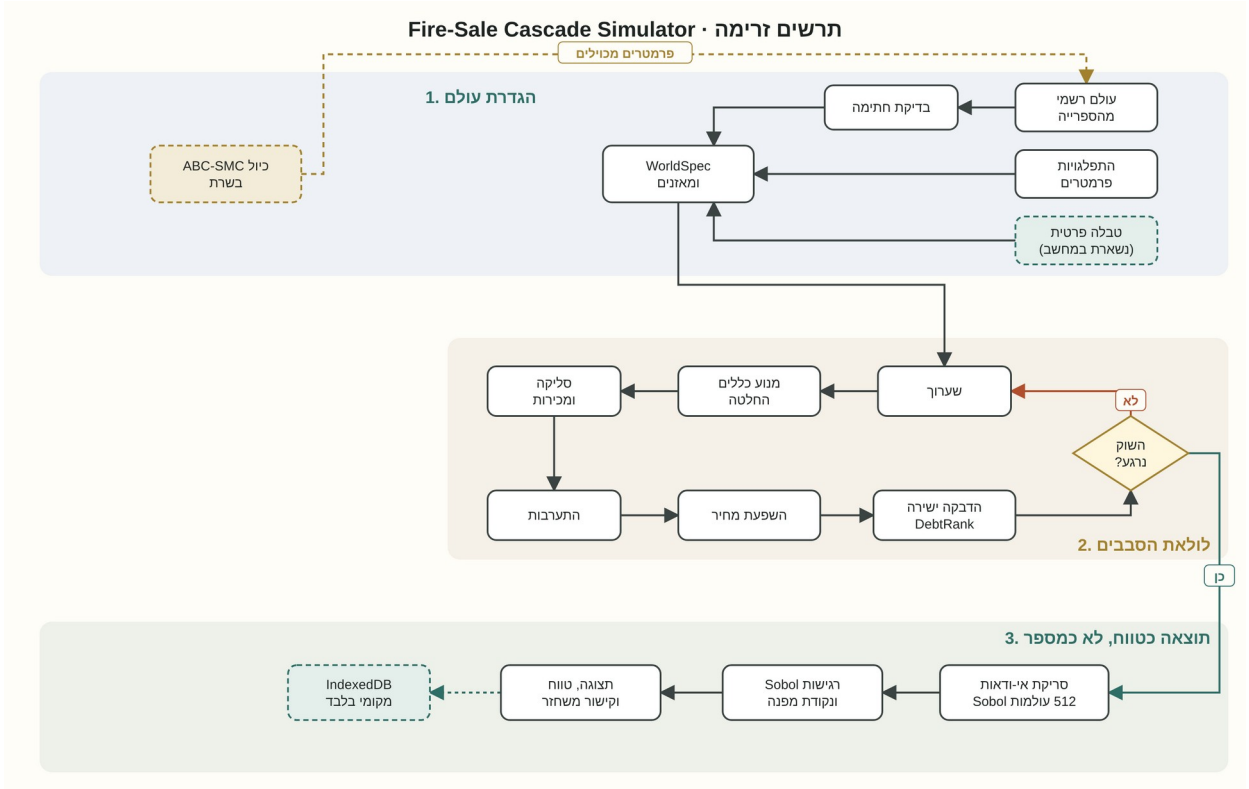
ספריות ופונקציות מרכזיות

שימוש	ספרייה	פעולה
forceSimulation, forceLink	d3-force	פריסת רשת
requestAnimationFrame	Canvas 2D	אנימציה
idb-keyval	IndexedDB	אחסון מקומי
'connect-src 'self	CSP	חסימת דליפה

טבלת טכנולוגיות

פיצ'ר	טכנולוגיות ושפות תכנות	משאבים ושירותים חיצוניים
1. בניית עולם	Python, Pydantic, NumPy, PyNaCl	ספריית עולמות חתומה (קבצים סטטיים)
2. מאזן ושערוך	NumPy, QuantLib	אין
3. כללי תגובה	YAML, jsonschema, numexpr	אין
4. השפעת מחיר	NumPy	נתוני נפח מסחר ציבוריים לברירות מחדל
5. מנוע המפל	NumPy, Numba, Pyodide	אין (רץ בדפדפן)
6. רשת והדבקה	NetworkX, SciPy sparse	אין
7. התערבות	Python (Protocol)	תיעוד רשמי של BoE והפד
8. סריקה ורגישות	SALib, SciPy qmc	אין
9. שחזור וכיול	pyabc, Dask, Parquet	דוחות ציבוריים של BoE, הפד ו-SVB
10. תצוגה וקישור	D3.js, Canvas, IndexedDB, CSP	CDN לקבצים סטטיים בלבד

תרשים זרימה



התרשים מתאר שלושה שלבים. בהגדרה (למעלה), המשתמש בונה עולם משלוש דרכים: עולם רשמי שהחתימה שלו נבדקת, התפלגויות, או טבלה פרטית שנשארת במחשב שלו. במנוע (באמצע), לולאת הסבבים: שערוך, החלטה, סליקה, התערבות, השפעת מחיר והדבקה, וחזרה לשערוך, עד שהשוק נרגע. בתוצאה (למטה), הרצה בודדת לא יוצאת לבד: היא תמיד עוברת בסריקת אי-הוודאות, ורק אז מוצגת עם הטווח, תרשים הרגישות ונקודת המפנה. הכיול (משמאל) רץ בנפרד בשרת, ומזין את העולמות הרשמיים בפרמטרים מכוילים.

מבנה בסיס הנתונים

למערכת שני מאגרים. בצד השרת: PostgreSQL לספרייה הציבורית (סוגי סוכנים, כללים, עולמות רשמיים, כיולים), שמתפרסמת כקבצים סטטיים חתומים. בצד המשתמש: IndexedDB להיסטוריית ההרצות, שלעולם לא עוזבת את הדפדפן. אין במערכת שום טבלה בשרת שמכילה נתוני מאזן של משתמש.

בצד השרת: טבלת agent_types

- **type_id:** מסוג VARCHAR(16), מפתח ראשי ('ldi', 'bank', 'pcf', 'insurer', 'mmf').
- **display_name:** מסוג VARCHAR, חובה.
- **balance_fields:** מסוג JSONB, חובה (שורות המאזן שקיימות לסוג).
- **default_rule_set:** מפתח זר ל-rule_sets, חובה.

טבלת rule_sets

- **rule_set_id:** מסוג VARCHAR, מפתח ראשי (למשל 'ldi_v1').
- **type_id:** מפתח זר ל-agent_types.
- **yaml_source:** מסוג TEXT, חובה.

- **sha256:** מסוג CHAR(64), חובה, ייחודי.
- **created_at:** מסוג TIMESTAMPTZ.

טבלת worlds

- **world_id:** מסוג CHAR(64), מפתח ראשי (SHA-256 של ההגדרה הקנונית).
- **name:** מסוג VARCHAR, חובה.
- **spec:** מסוג JSONB, חובה (WorldSpec מלא).
- **official:** מסוג BOOLEAN, ברירת מחדל false.
- **round_length_hint:** מסוג VARCHAR (למשל 'כחצי יום'), יכול להיות NULL.
- **calib_id:** מפתח זר ל-calibration_runs, יכול להיות NULL.

טבלת world_signatures

- **world_id:** מפתח זר ל-worlds, מפתח ראשי.
- **key_id:** מסוג VARCHAR, חובה.
- **signature:** מסוג BYTEA (64 בייט), חובה.
- **signed_at:** מסוג TIMESTAMPTZ, חובה.

טבלת world_sources

- **world_id:** מפתח זר ל-worlds.
- **field_path:** מסוג VARCHAR (למשל 'populations[0].leverage.mean'), חובה.
- **source_title:** מסוג VARCHAR, חובה.
- **source_url:** מסוג VARCHAR, חובה.
- **מפתח ראשי משולב:** (world_id, field_path).

טבלת calibration_targets

- **target_id:** מסוג SERIAL, מפתח ראשי.
- **case_id:** מסוג VARCHAR, חובה ('svb_2023', 'uk_ldi_2022').
- **metric:** מסוג VARCHAR, חובה.
- **value:** מסוג DOUBLE PRECISION, חובה.
- **tolerance:** מסוג DOUBLE PRECISION, חובה.
- **source_url:** מסוג VARCHAR, חובה.

טבלת calibration_runs

- **calib_id:** מסוג UUID, מפתח ראשי.
- **case_id:** מסוג VARCHAR, חובה.
- **method:** מסוג VARCHAR, חובה ('smm', 'abc_smc').
- **n_sims:** מסוג INTEGER, חובה.
- **accepted:** מסוג INTEGER, חובה.
- **posterior_path:** מסוג VARCHAR, חובה (קובץ Parquet).
- **engine_version:** מסוג VARCHAR, חובה.

טבלת fit_reports

- **calib_id:** מפתח זר ל-calibration_runs.
- **metric:** מסוג VARCHAR.
- **target, sim_p10, sim_p50, sim_p90:** מסוג DOUBLE PRECISION.
- **within_tolerance:** מסוג BOOLEAN, חובה.
- **מפתח ראשי משולב:** (calib_id, metric).

בצד המשתמש (IndexedDB): מאגר runs

- **run_id:** מפתח (SHA-256 של RunLink, או UUID לעולם פרטי).
- **created_at:** תאריך.
- **link:** אובייקט RunLink, או null לעולם פרטי.
- **private_world:** בוליאני.
- **result:** CascadeResult דחוס (סבבים כמערכים טיפוסיים).

מאגר sweeps

- **sweep_id:** מפתח.
- **run_id:** הפניה למאגר runs.
- **spec:** SweepSpec.
- **result:** SweepResult.

מאגר private_worlds

- **world_local_id:** מפתח (UUID).
- **name:** מחרוזת.
- **balances:** מערכים טיפוסיים של המאזנים.
- **הערה:** נמחק כולו בלחיצה אחת, ואין לו גיבוי בשרת.

מה האפיון סגר, ומה נשאר פתוח

- **נסגר:** המודל המלא מכסה את שני המנגנונים מהסיפור (מנוף ובטוחות, ריצת מפקידים), את שני ערוצי ההדבקה (מחירים וחובות ישירים), וארבעה סוגי התערבות שקרו באמת. כל ששת האיומים ממודל האיום קיבלו מענה: עולמות חתומים, טווח ורגישות בכל תוצאה, קישור משחזר, ריצה מקומית בלבד, מגבלת משאבים, והצהרה על מטרת הכלי.
- **נשאר פתוח:** ה-POC הראה, והאפיון לא משנה את זה, שהתוצאה תלויה בעיקר בנתוני חשיפה שאיננו רואים. פיצ'ר 8 יודע לומר בדיוק כמה התשובה תלויה בהם, ופיצ'ר 9 יודע לכייל על משברי עבר. אבל על השוק של היום, הכלי עדיין עונה "זה תלוי במנוף שלא ידוע לנו". הוא יודע לחשב מה יקרה, אם רק יודעים מי מחזיק מה.